Module 2FA — Authentification à double facteur

Guide d'intégration pour l'équipe de développement du site hôte

TOTP compatible Google Authenticator / Authy / FreeOTP — API REST Node.js autonome

Version du module : 1.0.0
Document rédigé le : 31 août 2026
Nature : API REST, indépendante du frontend

Sommaire

- 1. Présentation et périmètre
- 2. Principe d'intégration
- 3. Prérequis
- 4. Installation
- 5. Configuration (.env)
- 6. Base de données
- 7. Référence des endpoints
- 8. Parcours d'intégration côté site hôte
- 9. Sécurité — points impératifs
- 10. Codes d'erreur
- 11. Tests et outils fournis
- 12. Checklist de mise en production
- 13. Dépannage rapide

1. Présentation et périmètre

Le module ajoute une deuxième étape de connexion (code à 6 chiffres généré par une application d'authentification) à un site web existant, sans toucher à son système de connexion.

Ce que le module fait

- Génère un secret TOTP et le QR code à scanner dans Google Authenticator.
- Active / désactive le 2FA pour un utilisateur.
- Vérifie un code à 6 chiffres au moment de la connexion.
- Fournit des codes de récupération à usage unique (perte du téléphone).
- Chiffre les secrets en base (AES-256-GCM) et hache les codes de récupération (bcrypt).
- Applique un rate limiting sur les tentatives de vérification.

Ce que le module ne fait PAS

- Il ne gère pas la connexion (identifiant / mot de passe) ni les sessions : cela reste le rôle du site hôte.
- Il n'émet aucun jeton : il se contente de vérifier le JWT déjà produit par le site hôte.
- Il n'a pas d'interface graphique : c'est une API que le backend du site hôte appelle.

Pile technique

Composant	Détail
Exécution	Node.js >= 18, Express 4
Base de données	MySQL 5.7+ / 8.x (dédiée ou partagée avec le site hôte)
TOTP / QR	speakeasy, qrcode
Cryptographie	AES-256-GCM (secrets), bcrypt (codes de récupération)
Protection	express-rate-limit sur /verify, /disable, /recovery-codes/regenerate

2. Principe d'intégration

Le module se place derrière le backend du site hôte. Il ne parle jamais directement au navigateur.

```
Navigateur
   |  (identifiant + mot de passe, puis code 2FA)
   v
Backend du SITE HOTE    -- gere login, mot de passe, session, emet le JWT
   |  Authorization: Bearer <JWT de l'utilisateur>
   v
MODULE 2FA (cette API)  -- verifie le JWT, applique la logique TOTP
   |
   v
MySQL (tables two_factor_*)
```

Identification de l'utilisateur. Le module attend sur chaque requête le header Authorization: Bearer <jwt>. Ce JWT est celui que le site hôte émet déjà pour ses propres besoins. Le module le vérifie avec le même secret (JWT_SECRET) et lit l'identifiant de l'utilisateur dans le premier champ disponible du payload : sub, userId ou id.

> Le module ne signe aucun JWT. Il ne fait que les vérifier. Il faut donc lui communiquer la valeur de JWT_SECRET utilisée par le site hôte, et rien de plus.

Variante possible (non activée par défaut) : si le module est appelé par une API centrale plutôt que directement par le backend applicatif, l'authentification par JWT peut être remplacée par une clé d'API de service (header X-Api-Key) accompagnée de l'identifiant utilisateur (header X-User-Id). Demander cette variante si besoin.

3. Prérequis

- Node.js >= 18 et npm sur le serveur qui héberge le module.
- Une base MySQL accessible (base existante du site hôte, ou base dédiée).
- Le site hôte émet déjà des JWT signés en HS256, avec l'identifiant utilisateur dans sub, userId ou id.
- Communication HTTPS entre le backend hôte et le module (le JWT y circule).

4. Installation

1. Copier le dossier du module sur le serveur, par exemple dans /opt/2fa-google-authenticator.
2. Installer les dépendances (voir bloc ci-dessous).
3. Créer le fichier de configuration : cp .env.example .env puis éditer .env (section 5).
4. Créer la base (si dédiée) et jouer la migration SQL.
5. Démarrer le module.
6. Vérifier le log : « Module 2FA à l'écoute sur le port 4000 ».

```
cd 2fa-google-authenticator
npm install --production
cp .env.example .env

mysql -u <user> -p -e "CREATE DATABASE IF NOT EXISTS two_factor_auth;"
mysql -u <user> -p two_factor_auth < database/migrations/001_create_two_factor.sql

npm start          # production
npm run dev        # developpement (rechargement auto)
```

En production, lancer le module via un gestionnaire de processus (pm2, systemd, Docker) avec redémarrage automatique.

5. Configuration (.env)

Variable	Oblig.	Défaut	Description
PORT	non	4000	Port d'écoute du module.

DB_HOST	non	localhost	Hôte MySQL.
DB_PORT	non	3306	Port MySQL.
DB_USERNAME	non	root	Utilisateur MySQL.
DB_PASSWORD	non	(vide)	Mot de passe MySQL. S'il contient un #, l'encadrer de guillemets dans le .env.
DB_NAME	non	two_factor_auth	Nom de la base.
JWT_SECRET	OUI	—	Identique au secret JWT du site hôte. Le module refuse de démarrer sans.
ENCRYPTION_KEY	OUI	—	Clé AES-256 : 32 octets hexadécimaux (64 caractères). openssl rand -hex 32.
TOTP_ISSUER	non	MyApp	Nom affiché à côté du compte dans l'application d'authentification.
TOTP_WINDOW	non	1	Tolérance : nombre de pas de 30 s acceptés avant / après l'instant courant.
RECOVERY_CODES_COUNT	non	10	Nombre de codes de récupération générés à l'activation.

> ENCRYPTION_KEY est critique. Elle chiffre tous les secrets TOTP en base. La sauvegarder hors du serveur et ne jamais la modifier une fois en production : un changement rend tous les secrets illisibles et oblige chaque utilisateur à refaire l'activation.

Exemple de .env :

```
PORT=4000
DB_HOST=localhost
DB_PORT=3306
DB_USERNAME=app
DB_PASSWORD="motdepasse"
DB_NAME=two_factor_auth
JWT_SECRET=le-meme-secret-que-le-site-hote
ENCRYPTION_KEY=3f9c1d...   (64 caracteres hexadecimaux)
TOTP_ISSUER=MonSite
TOTP_WINDOW=1
RECOVERY_CODES_COUNT=10
```

6. Base de données

La migration database/migrations/001_create_two_factor.sql crée deux tables. Elles peuvent cohabiter avec les tables existantes du site hôte dans la même base.

Table	Colonnes	Rôle
two_factor_secrets	id, user_id (unique), secret_encrypted, enabled, created_at, updated_at	Un enregistrement par utilisateur. Secret TOTP stocké chiffré (AES-256-GCM). enabled passe à vrai après /enable.
two_factor_recovery_codes	id, user_id, code_hash, used_at, created_at	Un enregistrement par code de récupération. Code haché (bcrypt). used_at renseigné à la consommation (usage unique).

user_id est un VARCHAR(64) : y placer l'identifiant utilisateur du site hôte, tel qu'il apparaît dans le JWT.

7. Référence des endpoints

Préfixe commun : /api/2fa. Toutes les requêtes exigent le header Authorization: Bearer <jwt>. Corps en JSON. Réponses : succès { "success": true, ... } ; erreur { "success": false, "message": "..." }.

GET /api/2fa/status

Indique si le 2FA est actif pour l'utilisateur. Aucun corps.

```
200  { "success": true, "enabled": false }
```

POST /api/2fa/setup

Génère (ou regénère) un secret TOTP et le QR code. Le 2FA n'est pas encore actif après cet appel.

```
Corps   { "accountName": "utilisateur@example.com" }   // optionnel, sinon = user_id
200     { "success": true,
          "secret": "LMWGKNKYJQRUCSJGEQYGKNCNF5FFORKH",
          "qrCodeDataUrl": "..." }
```

qrCodeDataUrl est une image PNG encodée : l'injecter telle quelle dans <img src="..."> . secret peut être affiché pour une saisie manuelle dans l'application d'authentification.

POST /api/2fa/enable

Active le 2FA après vérification d'un premier code. Renvoie les codes de récupération.

```
Corps   { "token": "123456" }
200     { "success": true, "recoveryCodes": ["E63E-EBAC", "C372-C18C", ... 10 codes] }
401     { "success": false, "message": "Code invalide." }
404     { "success": false, "message": "Aucune configuration 2FA en attente pour cet utilisat
    eur." }
409     { "success": false, "message": "Le 2FA est deja active." }
```

> Les recoveryCodes ne sont renvoyés qu'ici (et à la regénération). Ils ne sont jamais réaffichables ensuite. À présenter une seule fois à l'utilisateur.

POST /api/2fa/verify

Vérifie un code au moment de la connexion. Accepte un code TOTP à 6 chiffres OU un code de récupération au format XXXX-XXXX. Rate limité.

```
Corps   { "token": "123456" }              // ou "E63E-EBAC"
200     { "success": true, "method": "totp" }
200     { "success": true, "method": "recovery_code" }
401     { "success": false, "message": "Code invalide." }
404     { "success": false, "message": "Le 2FA n'est pas active pour cet utilisateur." }
```

POST /api/2fa/disable

Désactive le 2FA et supprime la configuration. Exige un code valide (TOTP ou récupération). Rate limité.

```
Corps   { "token": "123456" }
200     { "success": true }
401     { "success": false, "message": "Code invalide." }
```

POST /api/2fa/recovery-codes/regenerate

Invalide les anciens codes de récupération et en génère de nouveaux. Exige un code valide. Rate limité.

```
Corps   { "token": "123456" }
200     { "success": true, "recoveryCodes": [ ... 10 nouveaux codes ] }
```

Rate limiting

/verify, /disable et /recovery-codes/regenerate sont limités à 5 requêtes par 15 minutes et par utilisateur. Au-delà :

```
429  { "success": false, "message": "Trop de tentatives, reessayez plus tard." }
```

8. Parcours d'intégration côté site hôte

Tous les appels ci-dessous sont faits par le backend du site hôte, en transmettant le JWT de l'utilisateur courant.

8.1 Activation du 2FA (utilisateur déjà connecté)

1. Backend hôte -> POST /api/2fa/setup. Récupère qrCodeDataUrl (et secret).
2. Le site affiche le QR code (<img src="{qrCodeDataUrl}">), plus le secret en clair pour la saisie manuelle.
3. L'utilisateur scanne dans Google Authenticator, lit le code à 6 chiffres et le saisit sur le site.
4. Backend hôte -> POST /api/2fa/enable avec { token }. Récupère recoveryCodes.
5. Le site affiche les codes de récupération une seule fois et invite à les sauvegarder.

```
// Exemple Node.js (backend du site hote)
const BASE = 'https://2fa.interne.monsite';

async function startSetup(userJwt) {
  const r = await fetch(`${BASE}/api/2fa/setup`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${userJwt}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ accountName: 'utilisateur@example.com' }),
  });
  return r.json();  // { secret, qrCodeDataUrl }
}

async function confirmSetup(userJwt, code) {
  const r = await fetch(`${BASE}/api/2fa/enable`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${userJwt}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ token: code }),
  });
  if (!r.ok) throw new Error('Code invalide');
  return r.json();  // { recoveryCodes: [...] }
}
```

8.2 Connexion avec 2FA

1. L'utilisateur saisit identifiant + mot de passe. Le site hôte les vérifie comme d'habitude.
2. Avant d'ouvrir la session : GET /api/2fa/status. enabled:false -> connexion normale ; enabled:true -> demander le code.
3. POST /api/2fa/verify avec { token } (code TOTP ou code de récupération). 200 -> ouvrir la session ; 401 -> refuser.

```
// Exemple Node.js - apres validation du mot de passe
async function loginSecondFactor(userJwt, code) {
  const status = await fetch(`${BASE}/api/2fa/status`, {
    headers: { 'Authorization': `Bearer ${userJwt}` },
  }).then(r => r.json());

  if (!status.enabled) return { ok: true };          // pas de 2FA

  const r = await fetch(`${BASE}/api/2fa/verify`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${userJwt}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ token: code }),
  });
  return { ok: r.ok };                               // true => ouvrir la session
}
```

8.3 Désactivation

Backend hôte -> POST /api/2fa/disable avec { token } (un code valide est exigé pour éviter une désactivation non consentie).

8.4 Régénération des codes de récupération

Backend hôte -> POST /api/2fa/recovery-codes/regenerate avec { token }. Les anciens codes cessent immédiatement de fonctionner. Afficher les nouveaux une seule fois.

9. Sécurité — points impératifs

- HTTPS entre le backend hôte et le module : le JWT et les codes y transitent.
- Ne pas exposer le module sur Internet. Le placer sur le réseau interne, joignable uniquement par le backend du site hôte.
- ENCRYPTION_KEY : secrète, sauvegardée hors serveur, jamais modifiée après mise en production.
- JWT_SECRET : strictement identique à celui du site hôte, stocké de façon sécurisée.
- Codes de récupération : hachés en base, non réaffichables. Présentés une seule fois.
- Rate limiting : déjà actif sur les vérifications (5 / 15 min / utilisateur). Ne pas le contourner côté hôte.
- Sauvegarde régulière des tables two_factor_secrets et two_factor_recovery_codes.
- Horloge serveur synchronisée (NTP) : le TOTP repose sur l'heure.

10. Codes d'erreur

HTTP	Message	Cause probable
401	Authentification requise.	Header Authorization absent ou mal formé.
401	Token invalide ou expiré.	JWT invalide - le plus souvent JWT_SECRET différent de celui du site hôte, ou JWT expiré.
401	Le token ne contient pas d'identifiant utilisateur.	Payload JWT sans sub / userId / id.
401	Code invalide.	Code TOTP faux/expiré, ou code de récupération inconnu / déjà utilisé.
400	Le champ token est requis.	Corps JSON sans token.
404	Aucune configuration 2FA en attente...	/enable appelé sans /setup préalable.
404	Le 2FA n'est pas activé pour cet utilisateur.	/verify ou /disable alors que le 2FA n'est pas actif.
409	Le 2FA est déjà activé.	/enable alors qu'il l'est déjà.
429	Trop de tentatives, réessayez plus tard.	Rate limit atteint (5 / 15 min / utilisateur).
500	Erreur interne du serveur.	Base injoignable, identifiants DB_* incorrects, ou ENCRYPTION_KEY mal formée.
404	Route introuvable.	URL hors du préfixe /api/2fa.

11. Tests et outils fournis

- Tests unitaires : npm test (13 tests, sans base de données requise).
- Collection Postman — parcours pas à pas : 2fa-sequence-activation-verification.postman_collection.json. 6 requêtes numérotées (statut -> setup -> enable -> statut -> verify -> verify par code de récupération). Le code à 6 chiffres est saisi à la main, comme un vrai utilisateur.
- Collection Postman — suite automatisée : 2fa-google-authenticator.postman_collection.json. Le JWT et le code TOTP sont calculés par les scripts de la collection ; exécutable avec newman run <fichier>.

Note : /verify, /disable et /regenerate étant limités à 5 appels / 15 min / utilisateur, un run complet de la collection automatisée consomme le quota. Pour rejouer dans la foulée, changer la variable userId de la collection.

12. Checklist de mise en production

- Node.js >= 18 installé sur le serveur cible.
- Base MySQL créée et migration 001_create_two_factor.sql jouée.
- .env renseigné.
- JWT_SECRET identique à celui du site hôte (vérifié).
- ENCRYPTION_KEY générée (openssl rand -hex 32) et sauvegardée hors serveur.
- TOTP_ISSUER = nom de la marque (visible dans l'application d'authentification).
- Module lancé via pm2 / systemd / Docker avec redémarrage automatique.
- Module joignable uniquement depuis le backend du site hôte (réseau interne / firewall).
- HTTPS en place (reverse proxy si nécessaire).
- Sauvegarde planifiée des tables two_factor_*.
- Horloge serveur synchronisée (NTP).
- Parcours activation puis connexion validés en recette.

13. Dépannage rapide

Symptôme	Piste
401 sur tous les appels, JWT pourtant valide côté site hôte	JWT_SECRET du module différent de celui du site hôte ; ou JWT expiré ; ou identifiant utilisateur absent du payload (sub / userId / id).
500 systématique	Voir les logs : Access denied ou ECONNREFUSED -> DB_* incorrects ; message sur ENCRYPTION_KEY -> elle doit faire 64 caractères hexadécimaux.
Le mot de passe MySQL contient # et la connexion échoue	Encadrer la valeur de guillemets dans le .env : DB_PASSWORD="mot#depasse".
Le QR code ne s'affiche pas	qrCodeDataUrl est déjà une image complète (...) : la mettre telle quelle dans src d'une balise <img>, sans la retraiter.
« Code invalide » alors que le code affiché semble correct	Horloge du serveur désynchronisée : activer NTP. En dépannage, augmenter TOTP_WINDOW à 2.
429 « Trop de tentatives »	Rate limit atteint : attendre 15 minutes, ou réduire le nombre de tentatives côté site hôte.
Après /enable, l'utilisateur n'a pas noté ses codes de récupération	Appeler /recovery-codes/regenerate (code valide requis) et réafficher les nouveaux codes une seule fois.

Fin du document — Module 2FA v1.0.0 — 31 août 2026.